Priority Queue : unordered list implementation

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.

A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority.

In our examples, we will assume that higher the numeric value, higher the priority.

Let us relook at the interface of the priority queue before discussing the implementation using unordered list.

Interface of the priority queue:

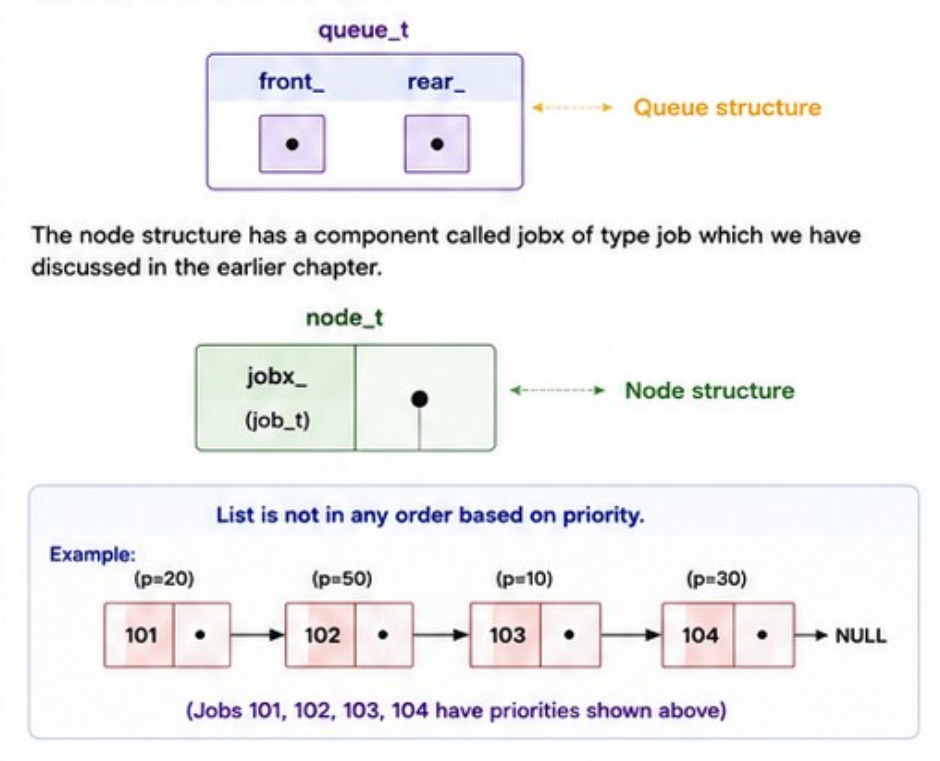
The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full

Implementation of priority queue:

The structure of the priority queue is same as that of queue using a linked list. It has two pointers to the node structure – one points to the beginning - called front and one points to the end – called the rear.

```
struct queue
{
        node_t* front_;
        node_t* rear_;
};
typedef struct queue queue_t;
```



The node structure has a component called jobx of type job which we have discussed in the earlier chapter.

```
struct node
{
        job_t jobx_;
        struct node* next_;
};
typedef struct node node_t;
```

The interface of priority queue remains same as that of queue. The implementation would dépend on the underlying data structure to support enque and deque appropriately.

```
void init_queue(queue_t *ptr_queue);
void deinit_queue(queue_t *ptr_queue);
void enque(queue_t *ptr_queue, job_t *ptr_job);
job_t deque(queue_t *ptr_queue);
int is_empty(queue_t *ptr_queue);
int is_full(queue_t *ptr_queue);
```

The list is not in any order based on priority. Let us discuss each one of these operations.

Initialize:

Both the front and the rear of the queue are grounded.

```
 void init_queue(queue_t *ptr_queue)
{
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Deinitialize :

This is one of the ways of removing all the elements in the queue. Deque each element until the queue becomes empty. As deque is a costly operation, we could do it better. Can you try it out?

Look at this code below. Do you think this is required?

```
ptr_queue->rear_ = ptr_queue->front_ = NULL;
```

```
void deinit_queue(queue_t *ptr_queue)
{
        while(! is_empty(ptr_queue))
        {
                deque(ptr_queue);
        }
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Is full :

This is a trick. Try allocating. If it succeeds, then there is memory available. Release the memory allocated just for the testing purpose.

```
int is_full(queue_t *ptr_queue)
{
   node_t* temp;
  temp = (node_t*)malloc(sizeof(node_t));
   if (temp == NULL) {
      return 1;
   }
   free(temp);
   return 0;
}
```

Is empty:

Check whether the front of the queue is NULL.

Can we check the rear of the queue in stead?

Would front and rear being equal indicate that the queue is empty?

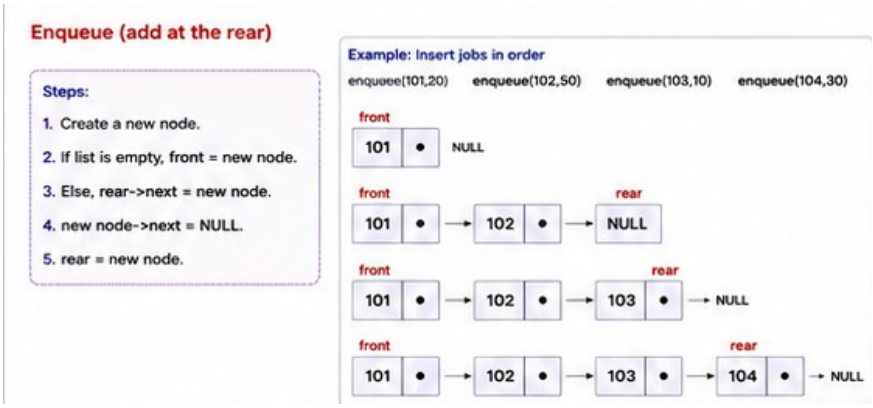
```
int is_empty(queue_t *ptr_queue)
{
        return ptr_queue->front_ == NULL;
}
```

Enque :

This operation does not consider the priority as the list is unordered. It adds the element at the end.

If queue is full, the program is exited.

Make a node. Populate the node. Add at the rear. Change front if the queue is empty. Otherwise make next of rear the new node added. Ground the next of the new node. Change the rear to point to the new node.



```
void enque(queue_t *ptr_queue, job_t *ptr_job)
{

	if(! is_full(ptr_queue))
	{
		node_t* temp = create_node(ptr_job);
		temp->next_ = NULL;
		if(ptr_queue->rear_ == NULL)
		{
			ptr_queue->front_ = temp;
		}
		else
		{
			ptr_queue->rear_->next_ = temp;
		}
		ptr_queue->rear_ = temp;
	}
	else
	{
		printf("queue full; cannot push\n");
	}
}
```

Deque :

We must find the node with the highest priority. If there are mode nodes with the same highest priority, we should select the job which came first. So, such a node will be closed to the front than the other nodes with the same priority.

If the queue is empty, the program is exited.

There are two parts in the algorithm.

The first is to find the position of the node with the maximum priority. We traverse the normal way.. We travel till the end of the list. We always keep track of the node with the highest priority – with two pointers – max_prev and max_pres.

The second part is to capture the job and then remove the node. If it is the first node, the front has to change. If it is the only node, the rear has to change. Otherwise we should make next of max_prev take the value of next of max_pres. Then release the max_pres node. Then return the job previously captured.

Do convince yourself that the order of operations matters.

A question for you – Can we remove the prev pointer from this algorithm?

```
job_t deque(queue_t *ptr_queue)

{

        if(! is_empty(ptr_queue))

        {

                // traverse

                node_t *prev = NULL;

                node_t *pres = ptr_queue->front_;

                int  max_priority = get_priority(&pres->jobx_);

                node_t *max_prev = NULL;

                node_t *max_pres = pres;

                int priority;

                while(pres != NULL)

                {

                        priority = get_priority(&pres->jobx_);
```

```
        if(priority > max_priority)
        {
                max_priority = priority;
                max_prev = prev;
                max_pres = pres;
        }

        prev = pres;
        pres = pres->next_;
}

job_t jobx;
jobx = max_pres->jobx_;

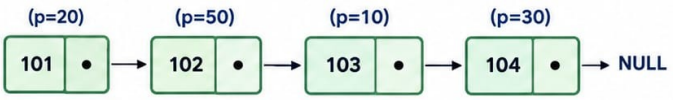
if(max_prev == NULL)
{
        ptr_queue->front_ = max_pres->next_;
        if(ptr_queue->front_ == NULL)
        {
                ptr_queue->rear_ = NULL;
        }
}
else
{
        max_prev->next_ = max_pres->next_;
        if(max_pres->next_ == NULL) // last node deleted
```

```
                {
                        ptr_queue->rear_ = max_prev;
                }
        }
        free(max_pres);
        return jobx;
    }
    else
    {
        printf("queue empty; cannot pop\n");
        exit(1);
    }
}
```

Dequeue (remove highest priority element)

Part 1: Find the node with highest priority

Keep track of the highest-priority node using two pointers: **max_prev** and **max_pres**.

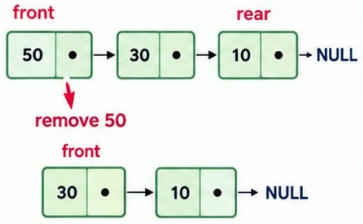


Example list

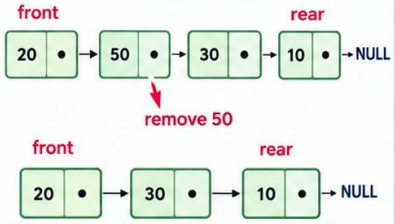
Step	pres (current)	max_prev	max_pres	Explanation
Start	101 (p=20)	NULL	101 (p=20)	Highest so far is 101
1	102 (p=50)	101	102 (p=50)	102 has higher priority
2	103 (p=10)	101	102 (p=50)	No change
3	104 (p=30)	101	102 (p=50)	No change

Part 2: Remove that node

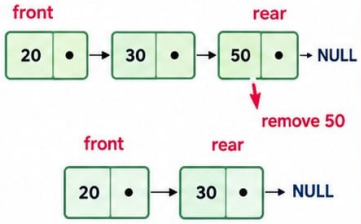
Case A: max_pres is front
(remove first node)



Case B: max_pres is in middle



Case C: max_pres is rear



Efficiency of operations :

Enque : This requires a few operations. We add at the rear without considering the priority.

Deque : This requires walking through the list to find the node with the highest priority. So, the number of operations is proportional to the number of nodes in the queue.